

Blockchain Module 2 Smart Contract & Ethereum

Maisonobe Alexandre
Teacher at MBDS and CTO at FusionBra

Introduction

Introduction

- Objectives of this video
- General definition of a smart contract
- How does it works

SmartContract Generalities

- Invented in 1994
- First Paper : Smart Contracts
(<http://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart.contracts.html>)
- By Nick Szabo

Smart Contract Generalitie

- Distributed Apps (Dapp) with no central server
- Leveraging the blockchain
- Strength
- « Wickness »
- A blockchain hosting blockchains

Ethereum Basis

Account, messages and transactions

Ethereum Basis

Ethereum components

- Ethereum Accounts
- Messages and Transactions
- The EVM & Scripts
- Blocks
- Speed
- Oracles
- Tokens
- Front-ends

Ethereum Basis – Ethereum Accounts

- Ethereum accounts (Externally owned / contract code)
- Nonce
- Ether balance
- Contract code
- Storage

Ethereum Basis – Messages and Transactions

➤ Transactions

- Recipient
- Signature
- Amount of account
- Optional data field
- STARTGAS
- GASPRICE

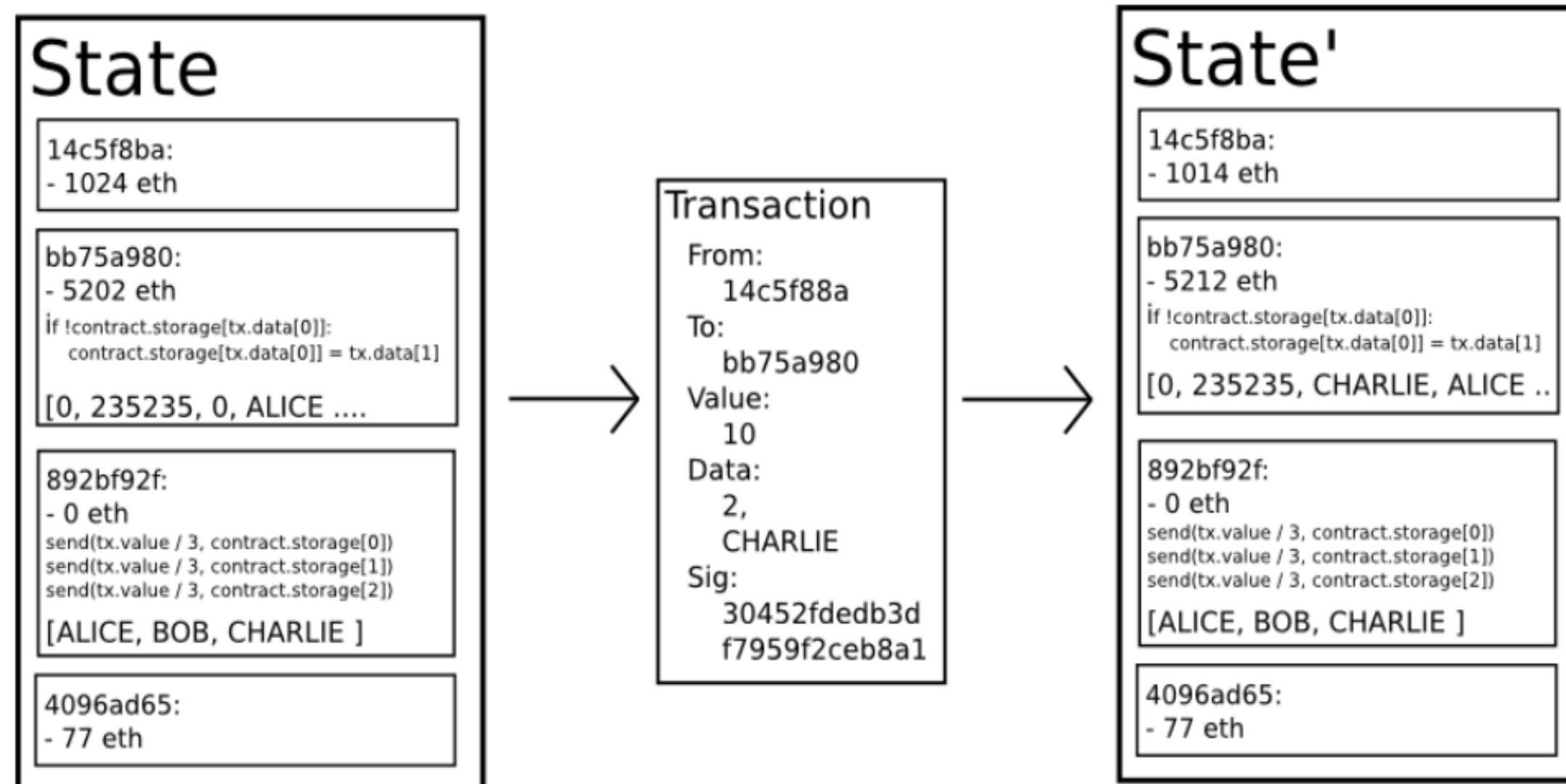
Ethereum Basis – Messages and Transactions

➤ Messages

- Recipient
- Signature
- Amount of account
- Optional data field
- STARTGAS

Ethereum Basis – Messages and Transactions

➤ State Transition



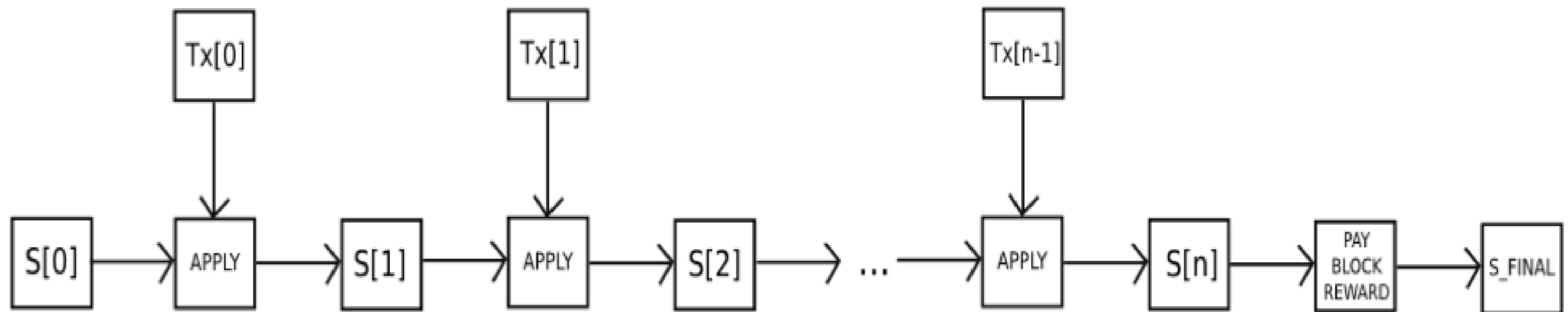
Ethereum Basis – Transaction Validation

Ethereum Basis – Transaction Validation

- State Transition
 - Check The transaction (validity, signature, nonce)
 - Calculates transaction fees ($\text{STARTGAS} * \text{GASPRICE}$)
 - Subtract fees
 - Increment Nonce
 - Initialize GAS counter and retrieve GAS per byte from the transaction
 - Perform the transaction $A \Rightarrow B$ (B can be a contract or an account)
 - Pays the fees to the miner and the change back to the sender

Ethereum Basis – Messages and Transactions

➤ State Transition



Ethereum Basis

EVM and Script

➤ EVM

- OPCODEs
- Turing Complete
- Sandbox

➤ Scripts (Solidity, Bamboo, Vyper)

```
pragma solidity ^0.4.21;

contract Coin {
    // The keyword "public" makes those variables
    // readable from outside.
    address public minter;
    mapping (address => uint) public balances;

    // Events allow light clients to react on
    // changes efficiently.
    event Sent(address from, address to, uint amount);

    // This is the constructor whose code is
    // run only when the contract is created.
    function Coin() public {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        if (msg.sender != minter) return;
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        if (balances[msg.sender] < amount) return;
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

Ethereum Basis – EVM & Script

- Address public minter ;
 - Declare a variable of address type
 - 160 bit value
 - Public makes it accessible outside the script , like a getter
- Mapping
(address => uint) public balances ;
 - Creates a public dictionary (Key/Values)
 - For an address a uint can be returned)
- Event Sent
(address from, address to, uint amount);
 - Declares an event applications can listen to

```
pragma solidity ^0.4.21;

contract Coin {
    // The keyword "public" makes those variables
    // readable from outside.
    address public minter;
    mapping (address => uint) public balances;

    // Events allow light clients to react on
    // changes efficiently.
    event Sent(address from, address to, uint amount);

    // This is the constructor whose code is
    // run only when the contract is created.
    function Coin() public {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        if (msg.sender != minter) return;
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        if (balances[msg.sender] < amount) return;
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

Ethereum Basis – EVM & Script

- Function `Coin() public{...}`
 - Constructor function called only when the contract is deployed
 - `Msg` is a global variable that can be accessed to get information from the sender
 - It stores in `minter` variable the address of the contract creator

```
pragma solidity ^0.4.21;

contract Coin {
    // The keyword "public" makes those variables
    // readable from outside.
    address public minter;
    mapping (address => uint) public balances;

    // Events allow light clients to react on
    // changes efficiently.
    event Sent(address from, address to, uint amount);

    // This is the constructor whose code is
    // run only when the contract is created.
    function Coin() public {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        if (msg.sender != minter) return;
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        if (balances[msg.sender] < amount) return;
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

Ethereum Basis – EVM & Script

- Function mint(...){...}
 - Verifies if the person calling this function is the minter
 - If true continues
 - Add the amount to the balance of the receiver
- Function send(...){...}
 - Check the funds
 - Change the balance
 - Invoke the event to notify the listeners

```
pragma solidity ^0.4.21;

contract Coin {
    // The keyword "public" makes those variables
    // readable from outside.
    address public minter;
    mapping (address => uint) public balances;

    // Events allow light clients to react on
    // changes efficiently.
    event Sent(address from, address to, uint amount);

    // This is the constructor whose code is
    // run only when the contract is created.
    function Coin() public {
        minter = msg.sender;
    }

    function mint(address receiver, uint amount) public {
        if (msg.sender != minter) return;
        balances[receiver] += amount;
    }

    function send(address receiver, uint amount) public {
        if (balances[msg.sender] < amount) return;
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);
    }
}
```

Ethereum Basis – EVM & Script

- Script memory management
 - The stack
 - Memory (quadratic cost)
 - Storage (dictionary)

- Msg
- Block
- tx

Ethereum Basis – EVM & Script

➤The price of storage

ZONE	EVM OPCODE	GAS/WORD	GAS/KB	GAS/MB
STACK	POP	2	64	65,536
	PUSHX	3	96	98,304
	DUPX	3	96	98,304
	SWAPX	3	96	98,304
MEMORY	CALLDATACOPY	3	98	2,195,456
	CODECOPY	3	98	2,195,456
	EXTCODECOPY	3	98	2,195,456
	MLOAD	3	96	98,304
	MSTORE	3	98	2,195,456
	MSTORE8	3	98	2,195,456
STORAGE	SLOAD	200	6,400	6,553,600
	SSTORE	20,000	640,000	655,360,000

Ethereum Basis – EVM & Script

➤ External Storage

➤ Swarm

- Swear
- Swindle

➤ FileCoin



Ethereum Basis

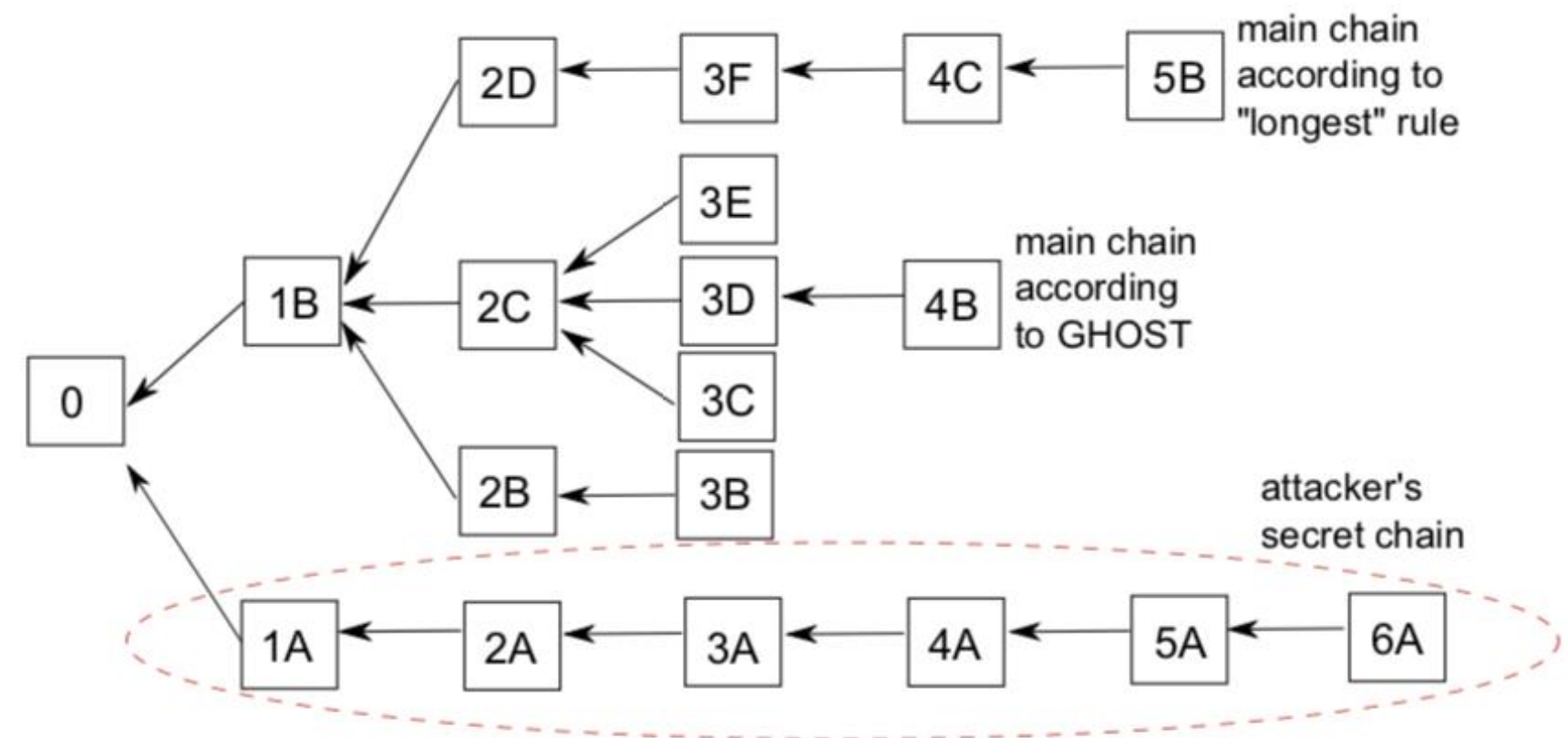
Blocks, oracles, tokens and frontend

➤ Block Building and Checking

- Check that the referenced block is the last one
- Check that the timestamp of the block is greater than that of the referenced previous block and less than 15 minutes into the future
- Check that the block number, difficulty, transaction root, uncle root and gas limit (various low-level Ethereum-specific concepts) are valid.
- Check that the proof of work on the block is valid.
- Let $S[0]$ be the state at the end of the previous block.
- Let TX be the block's transaction list, with n transactions. For all i in $0 \dots n-1$, set $S[i+1] = \text{APPLY}(S[i], \text{TX}[i])$. If any application returns an error, or if the total gas consumed in the block up until this point exceeds the GASLIMIT, return an error.
- Let S_FINAL be $S[n]$, but adding the block reward paid to the miner.
- Check if the Merkle tree root of the state S_FINAL is equal to the final state root provided in the block header. If it is, the block is valid; otherwise, it is not valid.

Ethereum Basis – Speed

- 1 block every 16 seconds
- GHOST (Greedy Heaviest Obscured Subtree)
- Includes Uncles on the calculation on the heaviest chain



https://blog.csdn.net/Vittle_stupid_child

Ethereum Basis – Oracles

- What is an Oracle
- How important are they ?
- An example

Ethereum Basis – Tokens

- What are Ethereum Tokens ?
- Fungible Vs Unfungible

Ethereum Basis – Frontend

➤ Web3

- Description
- Usage
- Tools (MetaMask , Mist , Parity)
- Api to access contracts

Let's make a Dapp

Overview

Some Ethereum Tools

- Truffle (A NodeJS developpment framework for ethereum)
- Ganache (A local Ethereum chain for tests)

Building a Dapp with Truffle and Ganache

➤ Setup

- Install Node JS (<https://nodejs.org/en/>)
- Install Truffle (`npm install -g truffle`)
- Install Ganache(<https://www.trufflesuite.com/ganache>)

- Create a folder for your exercise (`mkdir MoneyT / cd MoneyT`)
- Initialise a new truffle project (`truffle init`)
- Explore the architecture of the folder created

Building a Dapp with Truffle and Ganache

➤ Implement code

- Go to ./contracts and create MoneyT.sol
- Write the code we tested together earlier (see next slide)

Building a Dapp with Truffle and Ganache

```
pragma solidity ^0.5.16;
contract MoneyT{
    address public minter;
    mapping (address => uint) public balances;
    event Sent(address from, address to, uint amount);
    constructor() public { minter = msg.sender;}

    function mint(address receiver, uint amount) public {
        if (msg.sender != minter) return;
        balances[receiver] += amount; }

    function send(address receiver, uint amount) public {
        if (balances[msg.sender] < amount) return;
        balances[msg.sender] -= amount;
        balances[receiver] += amount;
        emit Sent(msg.sender, receiver, amount);}
}
```

Building a Dapp with Truffle and Ganache

- Prepare deployment file
 - Create ./migrations/2_deploy_contracts.js
 - Here is the code for it :
 - `var MoneyT = artifacts.require("./MoneyT.sol");`
 - `module.exports = function(deployer) {deployer.deploy(MoneyT);};`

Building a Dapp with Truffle and Ganache

- Run Ganache
 - Open your truffle project in Ganache
 - Check that Ganache port matches the one in your `./truffle-config.js`
- In your console
 - `truffle migrate`
 - `truffle console`
 - `MoneyT.deployed().then(function(instance) { app = instance })`

Building a Dapp with Truffle and Ganache

➤ You can now test your functions in truffle and follow the transactions in Ganache

- `app.minter()`
- `app.mint('0x.....A',50)`
- `app.mint('0x.....B',75)`
- `app.balances('0x.....A')`
- `app.send('0x.....B',20)`
- `app.balances('0x.....A')`
- `app.balances('0x.....B')`

EXPERIMENT !!!